

Какая информация используется в этой лабораторной работе

Краткая суть

Это веб-приложение — **интерактивный конвертер цвета**. Пользователь задаёт цвет в одной из трёх моделей (**RGB**, **CMYK**, **HSV**), а программа мгновенно пересчитывает его во все остальные и показывает результат на экране.

Какие данные использует программа

Входные данные (от пользователя)

Программа принимает цвет в трёх взаимозаменяемых формах:

Модель	Компоненты	Диапазон
RGB	R, G, B	0...255
CMYK	C, M, Y, K	0...100 %
HSV	H, S, V	H: 0...360°, S/V: 0...100 %

Каждую компоненту можно задать двумя способами:

1. **ползунком** — плавное изменение;
2. **числовым полем** — точное значение.

Плюс есть **HSV-палитра** (canvas), где цвет выбирается кликом мыши по квадрату насыщенность × яркость.

Промежуточные данные (внутренние)

Внутри программы цвет всегда хранится как **три числа R, G, B** (целые 0...255). Это «единый язык», через который проходят все пересчёты:

CMYK → RGB → HSV	HSV → RGB → CMYK
RGB → HEX	HEX → RGB

Такая схема называется **hub-and-spoke**: RGB — центральный узел, потому что через него проходят все остальные переводы.

Выходные данные (отображение)

Что показывается	Источник
Большое превью цвета	строка <code>rgb(R, G, B)</code>
Значения RGB	текущие <i>R, G, B</i>
Значения CMYK	результат <code>RGB → CMYK</code>
Значения HSV	результат <code>RGB → HSV</code>
Код HEX	перевод <i>R, G, B</i> в шестнадцатеричный вид
Маркер на палитре	текущие <i>S</i> и <i>V</i> для выбранного <i>H</i>

Какие знания из лекции здесь применяются

1. Классификация цветовых моделей

Модель	Класс	Область применения
RGB	аддитивная, аппаратно-зависимая	мониторы
CMYK	субтрактивная, аппаратно-зависимая	полиграфия
HSV	перцепционная (интуитивная)	выбор цвета человеком

2. Формулы преобразования (из лекции, стр. 36–48)

RGB ↔ CMYK:

$$K = \min\left(1 - \frac{R}{255}, 1 - \frac{G}{255}, 1 - \frac{B}{255}\right)$$

$$C = \frac{1 - \frac{R}{255} - K}{1 - K}, \quad \text{аналогично } M, Y$$

$$R = 255(1 - C)(1 - K), \quad \text{аналогично } G, B$$

RGB ↔ HSV:

$$V = \max(R', G', B'), \quad S = \frac{\max - \min}{\max}$$

$$H = 60^\circ \cdot \{ \dots$$

Обратно: через $C = VS$, $X = C(1 - |(H/60^\circ) \bmod 2 - 1|)$ и шесть секторов.

RGB ↔ HEX:

$$\text{HEX} = \text{hex}(R) \text{hex}(G) \text{hex}(B)$$

каждая компонента — два шестнадцатеричных разряда.

3. Законы Грассмана

1. **Трёхмерность** — цвет описывается ровно тремя числами.
2. **Аддитивность** — цвет смеси зависит только от компонент.
3. **Непрерывность** — малое изменение координаты даёт малое изменение цвета. Именно поэтому ползунки работают «плавно».

4. Цветовой охват

CMYK имеет **меньший** охват, чем RGB. Поэтому при $\text{RGB} \rightarrow \text{CMYK} \rightarrow \text{RGB}$ часть оттенков теряется. В программе это не заметно, потому что оба пересчёта идут через RGB без «сжатия» в CMYK — но в реальной печати эта проблема существует.

Какую информацию НЕ использует

1. Не используются физические спектры (длины волн).
2. Не используется модель **CIE XYZ** — она слишком тяжёлая для простого пикера.
3. Не используется **Lab** — нужна только для равноконтрастных сравнений.
4. Не используются **YIQ** и **YUV** — это телевизионные системы, не для GUI.

Итоговая мнемоника

Программа использует три вещи:
три модели (RGB, CMYK, HSV), три формулы (RGB↔CMYK, RGB↔HSV, RGB↔HEX), одно правило (всё считается через RGB).

Проблема «заблокированных палитр»

Цветовые модели RGB · CMYK · HSV
и паттерн «Наблюдатель»

Введение

Разбирается инцидент в приложении-пикере цвета: как только пользователь выставлял компоненты **С**, **М** и **У** в максимум (100 %), интерфейс «замирал» — ползунки и палитра HSV перестали давать видимый результат. Ниже описано, почему это происходило, как было исправлено и какое отношение к этому имеет паттерн проектирования «Наблюдатель».

Архитектура приложения как Observer

В коде нет класса `Observer`, но схема взаимодействия — это классический **Observer** без явного имени:

Роль	Что играет роль в коде
Subject (издатель)	Текущее значение цвета — по сути кортеж <code>(r, g, b)</code>
Observers (подписчики)	Ползунки RGB/CMYK/HSV, числовые инпуты, HEX-дисплей, превью, canvas палитры
notify	Три функции-рассылки: <code>updateFromRGB</code> , <code>updateFromCMYK</code> , <code>updateFromHSV</code>

Любой «наблюдатель» может изменить состояние: пользователь дёргает ползунок → вызывается соответствующая `updateFrom...` → она пересчитывает цвет и уведомляет всех остальных. То есть наблюдатель одновременно является и источником, и приёмником уведомлений.

Симптом

Последовательность, которую видит пользователь:

- Тянет ползунок **С** в 100 %.
- Ползунки **С**, **М**, **У** внезапно «отпрыгивают» на 0, а **К** — на 100.
- После этого ползунки **С**, **М**, **У** и вся HSV-палитра не дают никакого видимого эффекта — цвет «залипает» в чёрном.
- Живыми остаются только **К** и **V**.

Пошаговый разбор (trace)

Шаг	Действие	RGB	CMYK / HSV
0	старт	<code>(255, 77, 77)</code>	<code>(0, 70, 70, 0)</code> / <code>(0, 70, 100)</code>
1	пользователь ставит <code>C=M=Y=100</code>	<code>(0, 0, 0)</code>	<code>(100, 100, 100, 0)</code>
2	<code>rgbToCmyk(0, 0, 0)</code>	<code>(0, 0, 0)</code>	<code>k = 1</code> → деление на <code>(1-k) = 0</code> → <code>NaN</code> , спасает <code> 0</code> → <code>(0, 0, 0, 100)</code>
3	слайдеры CMYK синхронизируются	<code>(0, 0, 0)</code>	<code>(0, 0, 0, 100)</code>
4	<code>rgbToHsv(0, 0, 0)</code>	<code>(0, 0, 0)</code>	<code>h=0, s=0, v=0</code>

После шага 4 состояние системы — «чёрная дыра»: любой ввод проваливается обратно в ноль.

Корневые причины

Математическая (фундаментальная)

Это свойство самих моделей, а не баг:

- в CMYK при $K = 100\%$ множитель $(1 - K) = 0$, поэтому $\text{cmykToRgb}(c, m, y, 100) = (0, 0, 0)$ для **любых** c, m, y . Компоненты C, M, Y физически перестают влиять на результат;
- в HSV при $V = 0$ множитель $V = 0$, поэтому $\text{hsvToRgb}(h, s, 0) = (0, 0, 0)$ для **любых** h, s . Компоненты H и S перестают влиять на результат.

Отсюда и ощущение «блокировки»: ползунки двигаются, но цвет не меняется.

Программная №1 — NaN из `rgbToCmyk`

```
var c = (1 - gr - k) / (1 - k) || 0;
```

При $k = 1$ получается $0/0 = \text{NaN}$, и $\text{NaN} || 0$ молча даёт 0. Внешне это выглядит как «магия»: пользователь выставил $C=M=Y=100$, а получил $K=100$ и нули в C, M, Y.

Программная №2 — потеря H и S в ахроматической точке

`rgbToHsv` для чёрного/белого/серого возвращает $h = 0, s = 0$. Если бы код писал эти нули «как есть», пользователь, вернув яркость вверх, получил бы красный ($h = 0$) вместо исходного оттенка. То есть цвет «теряется» безвозвратно.

Программная №3 — каскад уведомлений (Observer)

Схема «наблюдатель → издатель → уведомить всех наблюдателей → наблюдатель снова пишет в издателя» потенциально бесконечна. Без защиты получаем либо заикливание, либо осцилляцию значений, либо (при взаимных ограничениях) полную блокировку ввода.

Исправление

1. Семафор `updating` — защита от повторного входа

js

```
var updating = false;

function updateFromRGB(r, g, b) {
  if (updating) return;
  updating = true;
  // ... запись во все слайдеры, инпуты, HEX, превью, canvas ...
  updating = false;
}
```

Это ядро исправления в терминах Observer: пока идёт одна рассылка, новые уведомления игнорируются. Каскад обрывается на первом же шаге.

2. Сохранение H и S при ахроматической точке

js

```
if (s === 0 || v === 0) {
  h = Number(hSlider.value) || 0;
}
if (v === 0) {
  s = Number(sSlider.value) || 0;
}
```

Что это даёт:

- при $V = 0$ (чёрный) H и S не сбрасываются, а «запоминаются»;
- когда пользователь поднимает V обратно, цвет возвращается с прежним оттенком и насыщенностью;
- HSV-палитра перестаёт быть «мёртвой» — из чёрной точки всегда можно выйти в любой цвет.

3. Защита от NaN в rgbToСмук

`|| 0` в строке `(1 - rp - k) / (1 - k)` `|| 0` не даёт NaN распространиться на слайдеры. Это не устраняет математику ($K=100$ по-прежнему поглощает C, M, Y), но сохраняет корректное числовое состояние интерфейса.

4. Клэмп значений в updateFromRGB

js

```
r = Math.min(255, Math.max(0, Math.round(r)));
g = Math.min(255, Math.max(0, Math.round(g)));
b = Math.min(255, Math.max(0, Math.round(b)));
```

Гарантирует, что наблюдатели никогда не получают значение вне диапазона — это тоже часть контракта «издателя» в Observer.

Связь с паттерном «Наблюдатель»

Паттерн Observer формально требует трёх вещей:

1. **Subject** хранит список подписчиков и умеет их оповещать.
2. **Observer** имеет метод `update(...)`, вызываемый при изменении.
3. Изменения идут **от субъекта к наблюдателям**, без обратной связи.

В нашем коде первые два пункта выполняются буквально: «субъект» — это текущее значение цвета, «подписчики» — все контролы, `update` — функции `updateFrom...`. А вот третий пункт в исходной версии был нарушен: наблюдатели писали обратно в субъект и тем самым запускали новый цикл оповещения.

Из этого вытекают два классических правила, реализованных в исправлении:

Проблема Observer	Как решено в коде
Циклические уведомления (observer → subject → observer → ...)	Флаг <code>updating</code> : пока идёт рассылка, новые уведомления отбрасываются
Потеря состояния (наблюдатель не получает достаточно данных)	Сохранение H и S: ахроматическая точка помнит оттенок и насыщенность
Распространение «отравленных» значений (NaN, выход за диапазон)	<code> 0</code> в CMYK-конверсии и <code>Math.min/Math.max</code> в <code>updateFromRGB</code>

Итог

«Блокировка палитр» — не один баг, а сумма трёх эффектов:

1. **Математического**: при $K=100$ и $V=0$ часть компонент перестаёт влиять на результат — это свойство CMYK и HSV, а не ошибка.
2. **Численного**: $0/0 \rightarrow \text{NaN}$ в `rgbToСмук` без защиты.
3. **Архитектурного**: каскад уведомлений в сети наблюдателей без семафора повторного входа.

Исправление бьёт по всем трём фронтам одновременно:

- флаг `updating` разрывает цикл уведомлений;

- сохранение H и S делает HSV-палитру «живой» даже после ухода в чёрную точку;
- `|| 0` и клэмп по диапазону защищают числовое состояние.

В терминах паттерна «Наблюдатель» это означает: издатель должен уведомлять подписчиков **идемпотентно и без повторного входа**, а подписчики — сохранять своё значимое состояние, даже когда формально оно равно нулю.